

Scaling Laws for Neural Language Models

<https://arxiv.org/abs/2001.08361>

Review and analysis by Alexei Finski, September 2026

Based on

- the assumption that the test loss is not substantially affected in a wide range of architectural variations,
 - the assumption of the generalization of the power-law form of relations across datasets,
 - the definition of an optimal training run based on the critical batch size, and
 - a simple empirical relation, where the test loss at convergence is parameterized by the number of parameters, assuming unlimited compute and data, as well as any batch size,
- the study provides a set of power-law relations for the estimation of
- the best achievable test loss given a fixed amount of compute, assuming training at the critical batch size, but without prior knowledge of the critical batch size,
 - the compute-efficient number of parameters for achieving the best test loss given a fixed amount of compute, under the same assumptions related to the critical batch size, and
 - the critical batch size, given the above estimate of the best achievable test loss.

The ability to estimate the critical batch size from the target test loss is arguably the finding that enabled this study. A set of additional relations are also provided. The study was conducted primarily on decoder-only Transformer models.

Summary of experiments establishing the basis for assumptions

Figures 5-7, 20, 21

Before performing the main experiments, it was important to establish that given a fixed number of parameters, architectural variations did not lead to a substantial change in the test

loss. In the case of Transformers, varying the number of layers would result in a different number of attention-based comparisons across the context positions. Establishing that varying the aspect ratio did not result in a substantial change in the test loss was prerequisite for parametrizing the test loss with the number of parameters. According to Table 1, only the compute-bound operations, resulting in matrix-matrix multiplication during training, were counted towards the number of parameters N , and the non-embedding parameters were excluded unless stated otherwise.

In the aspect ratio plot of Figure 5, each line was associated with a constant number of parameters. The aspect ratio was changed at constant N equal to 50M, 274M or 1.5B, while keeping $N \approx 12n_{layer}d_{model}^2$. Increasing n_{layer} and decreasing d_{model} increased the number of layers and successive applications of the attention mechanism, while decreasing the amount of information passed across the layers. With respect to the attention mechanism, increasing n_{layer} and decreasing d_{model} redistributed the parameters from fewer but larger key, query, value, and projection weight matrices, to more but smaller key, query, value, and projection weight matrices. According to Table 1, it appears that a single attention head was used.

Within a wide range of aspect ratios, the test loss did not increase substantially as a percentage of the lowest achieved test loss. The loss increase was measured relatively to the lowest achieved test loss separately for each number of parameters (50M, 274M or 1.5B). Although the resulting lines were displayed at similar levels close to 0%, their absolute test loss values would be at different levels.

It was not obvious that changes in the aspect ratio would not affect the test loss substantially. According to the decoder-only Transformer architecture used in this figure and primarily in this study, after each addition to the residual stream, there would be a layer norm. Regardless of

the changes in the model dimension, the input values would be scaled to 0 mean and variance 1, before any learnable adjustment and application of the next feedforward or attention layer.

As n_{layer} decreased beyond the range of d_{model}/n_{layer} ratios indicated in Figure 5, the test loss increased substantially for each number of parameters (50M, 274M or 1.5B). This is consistent with Figure 6, where the test loss was affected in models with a low number of layers as the number of parameters increased. This suggests that there may be an upper bound on the aspect ratio that needs to be considered to avoid substantial test loss changes due to architectural variations. This upper bound may be parametrizable with the number of parameters. A relation for such an upper bound on the aspect ratio was not provided in the study.

The effect of the attention mechanism on the test loss was further investigated by varying the number of attention heads and their dimensions that added up to d_{model} . This was achieved at a constant number of parameters of 25M and across model dimensions of 256, 512, 1024. These attention-related architectural variations did not result in a substantial increase in the test loss. The effect of variations of the feed forward layer was also investigated at a fixed number of parameters of 50M. The test loss did not change substantially for $d_{ff}/d_{model} \lesssim 8$, and was within 2% of the best achieved test loss.

In summary, having a relatively small number of layers in models with up to 1.5B parameters was sufficient. The test loss achieved with single-head attention was within 2% of the lowest test loss achieved with multi-head attention variants. Any other architectural rearrangements, including increasing the number of layers, did not have a substantial effect on the test loss.

The results in Figures 5 and 6 formed the basis for the assumption of the scaling laws

that architectural decisions in a wide range, including with respect to the attention mechanism, did not have a substantial effect on the test loss.

Notably, the later derived test loss, achieved by a compute-efficient training run given a fixed amount of compute was within 10% of the test loss that would be achieved if the same model (the same number of parameters) was trained to convergence. This estimated 10% difference was larger than the test loss differences within 2% due to architectural variations in Figure 5.

As stated above, there may be an upper bound on the aspect ratio that needs to be considered to avoid substantial test loss changes due to architectural variations as the number of parameters increases. There may also be bounds for the other ratios used in Figure 5 that need to be considered as the number of parameters increases.

For each point in Figure 6, models across different numbers of parameters up to approximately 1B and across different aspect ratios were trained to near convergence. **Excluding the embedding parameters from the number of parameters allowed the test loss performance to converge to a trend**, with the exception of models with less than 2 layers. These data provided the basis for excluding the embedding parameters from the number of parameter in the scaling laws.

One of the consequences of the experiments in Figures 5-6 was that the redistribution of the attention mechanism in a wide architectural range did not have a substantial effect on the test loss. A further investigation into if and how the attention mechanism provided an advantage was meaningful. In Figure 7, given the same number of parameters, Transformers outperformed LSTMs. This difference in performance was associated with a reduction in the test loss for the tokens later in the context (with indices above ~100 from the context beginning). If there was an upper limit on how many previous tokens provided useful

information, and such upper limit was lower than the length of the context (1024), then at some token index and all the token indices above it, the per-token test loss would plateau, because the useful information was bounded by a moving window that was smaller than the context. In the right plot of Figure 7, the pair of lines corresponding to the 400K-parameter Transformer and LSTM demonstrated the difference in processing the long context information after the token index of ~ 100 . The effect of the long context information was further quantified by the lines corresponding to the 2M-parameter LSTM, 3M-parameter Transformer, and 200M-parameter LSTM. The tokens with the indices above 1000 for the 2M- and 200M-parameter LSTMs reached the per-token test loss values of around 3 and 5 respectively. The 3M-parameter Transformer had similar per-token test loss values for the first ~ 50 tokens as compared to the 2M-parameter LSTM, but then reached the per-token test loss values of around 4 for the token indices above 1000. This effect could be attributed to better information processing over a longer context in Transformers.

Figure 21 shows that increasing the number of parameters in a Transformer model led to a greater improvement of the per-token test loss for the tokens with indices of 64 and higher, suggesting that a context larger than 1024 could be used in models with more than 1B parameters. Figure 20 shows that the per-token test loss was similar across the entire context during an early training phase with a fixed number of parameters of 774M, but the improvement was more substantial for the later tokens during a later training phase.

In summary, Figures 5, 6, 7, 20, and 21 describe a study of primarily the decoder-only Transformer architecture with the objective of identifying the architectural bounds within which architectural variations did not substantially change the test loss. The number of parameters across these figures was up to 1.5B. As the authors investigated architectural bounds, they identified that the effective use of attention required to be within a range of aspect ratios. Redistributing attention within this range of aspect ratios did not substantially change the test

loss. This led to an additional investigation of the attention mechanism that showed that i) Transformers had an architectural advantage over LSTMs relating to better information processing in the context size of 1024, and ii) the per-token loss did not plateau within this context suggesting that its length was not limiting. **The presented data also suggest that the architectural bounds, within which variations did not substantially change the test loss, may need to be scaled appropriately as the number of parameters increases. However, relations for scaling these architectural bounds were not provided.** Such relations would be useful in connection with the parameterizations presented later in the study. For example, knowing the minimal required number of layers would be useful after estimating the compute-efficient model size for a fixed compute budget.

Figures 8, 24

The previous experiments showed the invariance of the test loss with respect to architectural variations. These experiments were performed on the WebText2 dataset. The later experiments, establishing the scaling laws, were also performed on this dataset. To generalize the scaling laws and the underlying assumptions, a correspondence of the power-law form of relations needed to be demonstrated across different datasets.

Towards this objective, models with different numbers of parameters were trained on the WebText2 dataset in Figures 8 (left plot), and then evaluated on the test sets from the same and other datasets. It appears that these models were trained to convergence to compare the test loss values achieved when the number of parameters was limited. The test loss values were approximately on straight lines for each dataset. In Figure 8 (right plot), a number of models were trained to convergence, and a large model was trained such that the intermediate test loss values during its training overlapped with test loss values of the smaller models trained to convergence. When evaluated on a different data distribution, the test loss values of the

smaller models were still approximately on the straight line of the intermediate test loss values of the large model. In Figure 24, different aspect ratios at a fixed number of approximately 1.5B parameters did not result in substantial changes of the test loss across data distributions (i.e. maintaining the same approximate constant offset from the data distribution used for training). The results of these three experiments in Figures 8 and 24 support the correspondence of test loss relations across different datasets within the power-law form, and provide the basis for the generalization argument for the power-law relations of the scaling laws. **Based on this generalization argument, a more limited number of training runs were necessary to establish the scaling laws.**

There is an additional generalization argument that does not appear to be clearly stated in the paper. The relations in the scaling laws involving compute-efficient training, relied on the definition of an optimal training run based on the critical batch size. The relation between the critical batch size and the noise scale was demonstrated by McCandlish et al. across tasks and models (see the comments on the theoretical interpretation of the scaling laws). These previous results suggest that the definition of an optimal training run, used for the purposes of the scaling laws, was generally applicable across tasks.

Summary of experiments establishing the scaling laws

Figure 1

The parameterization of the test loss under the conditions when the number of parameters was limited and the amount of data and compute were not limited, was obtained by fitting the following equation to the test loss values achieved by **training to convergence** across different numbers of parameters and at the batch size of 2^{19} .

$$L(N) = \left(\frac{N_c}{N}\right)^{\alpha_N}$$

The parameterization of the test loss under the conditions when the amount of data was limited and the number of parameters and the amount of compute were not limited, was obtained by fitting the following equation to the test loss data of a large model with $n_{layer} = 36$, and $d_{model} = 1280$, **trained until the test loss stopped decreasing** (i.e. it converged or increased due to overfitting) on fixed subsets of the WebText2 dataset and at the batch size of 2^{19} . The number of parameters was chosen such that it was not limited for the purposes of this experiment.

$$L(D) = \left(\frac{D_c}{D}\right)^{\alpha_D}$$

$L(N)$ and $L(D)$ had no assumption of training optimality. For these relations, compute was unlimited. These relations did not depend on the batch size, including the critical batch size, which defined an optimal training run. This is important for $L(N)$, to which the later derivation of relations involving compute-efficient training was reduced. In this later derivation the assumption was made that training was at the critical batch size, but the value of the critical batch size was unknown. This derivation worked because $L(N)$ was valid for training at any batch size, including the critical batch size.

The compute plot in Figure 1 shows training runs at the batch size 2^{19} , which was not the critical batch size for the different test loss levels. A parametrization of the test loss with respect to fixed compute, and under the assumption of compute-efficient training, was derived later.

Figure 9

$L(N)$ and $L(D)$ provided the test loss estimations i) under the assumption of unlimited compute and ii) under similar early stop conditions. This motivated the test loss parameterization $L(N, D)$ for estimating the test loss at the point of convergence or at the point when the test loss stopped decreasing, and without prior knowledge of whether N or D was limited. Under the assumption of unlimited compute, $L(N, D)$ provided an estimate of the best achievable test loss for any given pair of N and D , while also controlling for overfitting due to the early stop conditions.

A number of models with different numbers of parameters were trained (with 10% dropout) to convergence or until the test loss stopped decreasing across dataset sizes. The obtained test loss values, shown in Figure 9, were then fit to the following equation.

$$L(N, D) = \left[\left(\frac{N_c}{N} \right)^{\frac{\alpha_N}{\alpha_D}} + \frac{D_c}{D} \right]^{\alpha_D}$$

This parameterization of the early stopped test loss provided additional advantages. As

$N \rightarrow \infty$, $L(N, D)$ approached $L(D)$, and as $D \rightarrow \infty$, $L(N, D)$ approached $L(N)$.

Additionally, α_N and N_c could be fit separately when $D \rightarrow \infty$, and α_D and D_c could be fit separately when $N \rightarrow \infty$, instead of fitting all four parameters at the same time. However, in Figure 9 all four parameters were fit at the same time.

If $L(N, D) = L(D = \infty)$ and therefore $L(N, D)/L(D = \infty) - 1 = 0$, the lowest test loss for a given number of parameters was fully realized, because training was stopped at

convergence. If $L(N, D)/L(D = \infty) - 1 > 0$, the lowest test loss for a given number of parameters was not fully realized due to limited data, because training was stopped when the test loss stopped decreasing due to limited data (i.e. it may have briefly increased and the preceding value was used). As stated previously, $L(N, D)$ provided a test loss estimate without prior knowledge of whether N or D was limited.

The right plot of Figure 9, visualized the extent of the unrealized test loss due to limited data across different numbers or parameters and different amounts of data. The N and D values in the left plot were represented as the $N^{\frac{\alpha_N}{\alpha_D}}/D$ ratio in the right plot. As the $N^{\frac{\alpha_N}{\alpha_D}}/D$ ratio increased, the unrealized test loss due to limited data also increased. In the reviewed version of the paper, this plot was titled as “overfitting” and was also referred to as “extent of overfitting”. It is unclear why this title was used. This plot shows the training runs with the unrealized test loss due to limited data and the extent of this unrealized test loss. When data was limited, a training run was stopped early when the test loss stopped decreasing, and overfitting was controlled before the excessive parameter capacity resulted in the fitting of the noise and/or spurious patterns that could affect the model generalization. Additionally, compute-efficient training was later performed with models with more parameters than necessary, and under the compute-efficient early stopping conditions given a fixed compute budget.

Let’s assume that the test loss was limited by the number of parameters and the amount of data at the same time. In other words, it converged due to a limited number of parameters and it stopped decreasing due to a limited amount of data at the same value. At this test loss value, $L(D) = L(N)$ and the ratio between N and D is as follows.

$$\left(\frac{N_c}{N}\right)^{\alpha_N} = \left(\frac{D_c}{D}\right)^{\alpha_D}$$

$$\frac{N^{\alpha_N}}{D^{\alpha_D}} = \frac{N_c^{\alpha_N}}{D_c^{\alpha_D}}$$

$$\left(\frac{N^{\alpha_N}}{D^{\alpha_D}}\right)^{1/\alpha_D} = \left(\frac{N_c^{\alpha_N}}{D_c^{\alpha_D}}\right)^{1/\alpha_D}$$

$$\left(\frac{N^{\frac{\alpha_N}{\alpha_D}}}{D}\right) = \left(\frac{N_c^{\frac{\alpha_N}{\alpha_D}}}{D_c}\right), \text{ where the right side is constant.}$$

Across different values of N and D , there is a subset of values that match the above constant ratio. If N and D values match this ratio, then both N and D would need to be increased together in order to obtain an additional improvement of the test loss under the early stopping conditions for $L(N, D)$. If N and D values do not match this ratio, then increasing either N or D , depending on the value of the ratio, would be sufficient to obtain an additional improvement of the test loss.

In the right plot of Figure 9, the y axis compares the test loss $L(N, D)$ to the test loss with unlimited data $L(D = \infty)$, where $L(D = \infty)$ is limited by the number of parameters.

Therefore, when $L(N, D) = L(D = \infty)$, the number of parameters is limited. The constant

$\frac{N_c^{\frac{\alpha_N}{\alpha_D}}}{D_c}$ value is around the largest ratio value that is still close to zero on the y axis in terms of

$$L(N, D)/L(D = \infty) - 1.$$

Figure 10, 18

McCandlish et al. previously defined the noise scale B_{noise} and the simple noise scale B_{simple} for estimating the critical batch size B_{crit} . This estimation was possible because they measured the relation $B_{crit} \approx B_{simple}$ across performance levels (training loss), models, and tasks.

Measuring B_{noise} or B_{simple} required the gradients at an optimization step. This approach was not practical for the purposes of planning an expensive training run and estimating the critical batch size at a target loss for such a training run.

Instead, B_{crit} was estimated directly from the test loss in the present study. The authors stated that “we are not especially confident in the prediction of $B_{crit}(L)$ for values of the loss far outside the range we have explored”. However, estimating B_{crit} from the test loss was scalable, and B_{crit} was required by the definition of an optimal training run.

It appears that the same two-stage grid search procedure was performed to measure B_{crit} , as in the previous study by McCandlish et al. Briefly, a set of batch sizes was chosen, a set of learning rate schedules was identified for each batch size, a set of test loss performance levels was identified, and across the resulting training runs, the number of processed tokens E and the number of steps S were plotted for each batch size from the training run that achieved a performance level most quickly. Different learning rate schedules could be optimal for the same batch size but different levels of performance. The resulting Pareto fronts were fit to the following equation, where each fit was with respect to a level of performance.

$$\left(\frac{S}{S_{min}} - 1\right)\left(\frac{E}{E_{min}} - 1\right) = 1$$

B_{crit} was then obtained from the fit of the above equation at each level of performance. The entire procedure was performed for Transformer models with 3M and 85M parameters, as shown in Figure 18. The obtained B_{crit} values across the test loss values and across these two numbers of parameters (B_{crit} depended on the number of parameters only through the achieved test loss), were plotted against the noise scale in Figure 10. These B_{crit} values were fit to the following equation, and this fit was used for estimating B_{crit} given a target test loss value in the derivations of subsequent relations.

$$B_{crit}(L) = \frac{B_*}{L^{1/\alpha_B}}$$

The measurements of the noise scale required gradients from the training set at an optimization step. In Figure 10, it appears that the B_{crit} values were measured from the fits at pre-defined test loss levels with regular intervals, and were then plotted according to the corresponding training loss values. The noise scale measurements were made from the training set gradients and were more frequent.

In the previous study, McCandlish et al. mostly used a procedure that relied on data parallelism to estimate B_{simple} from a training set batch at an optimization step. In contrast, measuring B_{noise} required gradients across the entire training set at an optimization step. It appears that B_{noise} was measured in Figure 10.

Figure 11

For the purposes of the following derivation, it is helpful to distinguish between a test loss value L , achieved during optimal or non-optimal training, and a target loss value L_{target} , at which the critical batch size B_{crit} is estimated for optimal training.

Let's assume that a training run is performed with a batch size B that may not be the critical batch size. A number of steps S have been executed. S can be mapped to the corresponding number of steps S_{min} , theoretically performed as if training with a batch size $B \gg B_{crit}$. S_{min} can be estimated as follows.

$$\left(\frac{S}{S_{min}} - 1\right)\left(\frac{E}{E_{min}} - 1\right) = 1$$

$$\left(\frac{S}{S_{min}} - 1\right)\left(\frac{BS}{B_{crit}S_{min}} - 1\right) = 1$$

$$\frac{(S - S_{min})B_{crit}}{S_{min}B_{crit}} = \frac{S_{min}B_{crit}}{BS - B_{crit}S_{min}}$$

$$BS^2B_{crit} - SB_{crit}^2S_{min} - S_{min}B_{crit}BS + S_{min}^2B_{crit}^2 = S_{min}^2B_{crit}^2$$

$$BS^2B_{crit} - SB_{crit}^2S_{min} - S_{min}B_{crit}BS = 0$$

$$S_{min}(SB_{crit}^2 + SB_{crit}B) = BS^2B_{crit}$$

$$S_{min} = \frac{BS^2B_{crit}}{SB_{crit}(B_{crit} + B)} = \frac{BS}{B_{crit} + B} = \frac{S}{\frac{B_{crit}}{B} + 1}$$

The above expression for S_{min} depends on the target loss value L_{target} , at which B_{crit} is

estimated with $B_{crit}(L_{target}) = \frac{B_*}{L_{target}^{1/\alpha_B}}$. If $B = B_{crit}$ then $S_{min} = S/2$. If $B \gg B_{crit}$, then S

approaches S_{min} .

Under the assumption of unlimited data, the test loss can be limited by the number of parameters due to convergence, or by the number of steps if the test loss value at convergence was not reached. The above estimation of S_{min} enables the following test loss parameterization with the number of parameters N and the actual number of steps S , without prior knowledge of whether N or S is limited.

$$L(N, S) = \left(\frac{N_c}{N}\right)^{\alpha_N} + \left(\frac{S_c}{S_{min}(S)}\right)^{\alpha_S}$$

The above equation is valid for any N and S , and therefore does not assume a fixed compute budget. It is also valid for any B , and does not assume training at a critical batch size. But an estimation of the critical batch size at some target loss value L_{target} is necessary to obtain $S_{min}(S)$. Therefore, this equation requires a theoretical reference to an optimal training run.

In the right plot of Figure 11, models with different numbers of parameters were trained with $B = 2^{19}$ on unlimited data. The test loss values at fixed numbers of steps were fit to the equation for $L(N, S)$. It is unclear which value of L_{target} was used for the theoretical reference to an optimal training run in order to obtain $S_{min}(S)$. B was the same for all training runs. If the

same L_{target} was used across all the training runs, every S would map to $S_{min}(S)$ through the same theoretical reference. Alternatively, the best achieved test loss at each fixed S (from the model with the largest number of parameters) could be used as L_{target} . In this case, each fit line would have a different theoretical reference for $S_{min}(S)$. As stated previously, $L(N, S)$ provided a test loss estimate without prior knowledge of whether N or S was limited. The test loss at small N appeared to be limited by N , resulting in convergence as S increased. In contrast, the test loss at large N appeared to be limited by S before reaching convergence. The fit(s) to the equation for $L(N, S)$ were also shown.

In summary, $L(N, S)$ provided an estimate of the test loss, given a number of parameters and a number of steps. This estimate is valid for any fixed batch size, but requires a theoretical reference to an optimal training run with the critical batch size estimated at some target loss value.

Analysis of the estimation of the optimal compute budget allocation

Given a fixed amount of compute C , we are interested in describing an optimal training run that achieves the best test loss after consuming C . An optimal training run is performed with the critical batch size, estimated at the test loss L_{target} that this training run achieves after consuming C . An optimally compute-efficient (or compute-efficient) training run is an optimal training run that achieves the best test loss value after consuming C . The test loss value, achieved by a compute-efficient training run is assumed to be equal to L_{target} for estimating B_{crit} of this training run. B_{crit} does not depend on the number of parameter. For the same B_{crit}

and test loss, different amounts of compute could be consumed with different numbers of parameters. Therefore, the model used in a compute-efficient training run has a compute-efficient number of parameters $N_{eff}(C)$ or N_{eff} . The number of steps taken in a compute-efficient training run to achieve the best test loss after consuming C is $S_{eff}(C)$ or S_{eff} .

For compute-bound operations, resulting in matrix-matrix multiplication during training, and for relatively small context size used in this study (1024), the number of FLOPs per token was estimated according to Table 1 as $C_{forward} + C_{backward} \approx 2N + 4N = 6N$.

In the previous section, $L(N, S)$ provided an estimate of the test loss at any fixed batch size, but with a theoretical reference to an optimal training run at the critical batch size estimated at some target loss value. Therefore, $L(N, S)$ is also valid for optimal training at $B = B_{crit}$ where B is equal to the critical batch size of the theoretical reference, estimated at the target loss L_{target} . Substituting B_{crit} for B in $L(N, S)$ provides the following equation.

$$L(N, S) = \left(\frac{N_c}{N}\right)^{\alpha_N} + \left(\frac{S_c}{S_{min}(S)}\right)^{\alpha_S}$$

$$L(N, S) = \left(\frac{N_c}{N}\right)^{\alpha_N} + \left(\frac{2S_c}{S}\right)^{\alpha_S}$$

As noted earlier, $S_{min} = S/2$, if $B = B_{crit}$. Substituting the estimate $S \approx C/(6NB_{crit})$ given a

fixed amount of compute C , and the estimate $B_{crit}(L_{target}) = \frac{B_*}{L_{target}^{1/\alpha_B}}$, provides the following

equations.

$$L(N, C) \approx \left(\frac{N_c}{N}\right)^{\alpha_N} + \left(6NB_{crit}\frac{2S_c}{C}\right)^{\alpha_S}$$

$$L(N, C) \approx \left(\frac{N_c}{N}\right)^{\alpha_N} + \left(12NB_*\frac{S_c}{L_{target}^{1/\alpha_B}C}\right)^{\alpha_S}$$

The value L_{target} is only for a theoretical reference in $L(N, S)$. In contrast, the theoretical reference and the actual training run are the same in $L(N, C)$, because $B = B_{crit}$. But the amount of compute C is still disconnected from L_{target} . $L(N, C)$ simply estimates the test loss given a number of parameters and a fixed amount of compute, **assuming training at the critical batch size, estimated at some L_{target} that may not be equal to $L(N, C)$** . For example, L_{target} could be much lower than $L(N, C)$. In this case, $L(N, C)$ would simply be an intermediate loss obtained after consuming C , as if training towards L_{target} at the batch size B_{crit} .

As the next step, the **assumption is made that $L_{target} = L(N, C)$** . As a consequence, for any number of parameters N and fixed amount of compute C , the critical batch size is estimated at the test loss achieved after consuming C . Under this assumption, different values of N and C may result in different critical batch sizes B_{crit} . In contrast to $L(N, S)$, where B_{crit} of the theoretical reference was arbitrarily set through the choice of L_{target} , $L(N, C)$ dynamically sets B_{crit} depending on N and C . This assumption provides the following equation.

$$L(N, C) \approx \left(\frac{N_c}{N}\right)^{\alpha_N} + \left(12B_*S_c \frac{N}{L(N, C)^{1/\alpha_B}C}\right)^{\alpha_S}$$

Taking the first derivate with respect to N and setting it to zero, limits N to the compute-efficient number of parameters $N_{eff}(C)$ or N_{eff} , which is the number of parameters, at which the best test loss is achieved given a fixed amount of compute C . The best test loss value and the critical batch size, estimated at this test loss value, remain unknown at this point.

$$\frac{\partial L(N, C)}{\partial N} = 0$$

$$0 \approx -\alpha_N N_c^{\alpha_N} N^{-(\alpha_N+1)} + \alpha_S \left(\frac{12B_*S_c}{C}\right)^{\alpha_S} \left(\frac{N}{L(N, C)^{1/\alpha_B}}\right)^{\alpha_S-1} \left(\frac{1}{L(N, C)^{1/\alpha_B}} + 0\right)$$

$$\frac{\alpha_N}{\alpha_S} \left(\frac{N_c}{N}\right)^{\alpha_N} \approx \left(\frac{12B_*S_c}{C}\right)^{\alpha_S} \left(\frac{N}{L(N, C)^{1/\alpha_B}}\right)^{\alpha_S}$$

$$\frac{\alpha_N}{\alpha_S} \left(\frac{N_c}{N}\right)^{\alpha_N} \approx \left(12B_*S_c \frac{N}{L(N, C)^{1/\alpha_B}C}\right)^{\alpha_S}$$

Substituting the above expression in the equation for $L(N, C)$ is only valid for $N = N_{eff}$

$$L(N, C) \approx \left(\frac{N_c}{N}\right)^{\alpha_N} + \left(12B_*S_c \frac{N}{L(N, C)^{1/\alpha_B}C}\right)^{\alpha_S}$$

$$L(N_{eff}, C) \approx \left(\frac{N_c}{N_{eff}}\right)^{\alpha_N} + \frac{\alpha_N}{\alpha_S} \left(\frac{N_c}{N_{eff}}\right)^{\alpha_N}$$

$$L(N_{eff}, C) \approx \left(1 + \frac{\alpha_N}{\alpha_S}\right) \left(\frac{N_c}{N_{eff}}\right)^{\alpha_N}$$

The subexpression $\left(\frac{N_c}{N_{eff}}\right)^{\alpha_N}$ is of interest. If we consider $N_{eff}(C)$ as simply some value of the

number of parameters, disregarding its relation to compute, then this subexpression estimates the test loss obtained as if a model with N_{eff} parameters was trained to convergence with

unlimited compute and data (see Figure 1). **As stated previously, training to convergence given a limited number of parameters and unlimited compute and data, does not depend on the batch size and therefore is valid for training at the critical batch size estimated at**

$L(N_{eff}, C)$. The values of the exponents α_N and α_S were previously determined and

$\frac{\alpha_N}{\alpha_S} \approx 10\%$. Therefore $L(N_{eff}, C)$ is within 10% of the test loss obtained if the same model

with N_{eff} parameters would be trained to convergence with unlimited compute and data.

$L(N_{eff}, C)$ provides an estimate of the test loss obtained with compute-efficient training given

a fixed amount of compute C , but requires N_{eff} as input.

The above equations for $L(N_{eff}, C)$ enable estimating the compute-efficient number of

parameters N_{eff} , given a fixed amount of compute C , as follows.

$$L(N_{eff}, C) \approx \left(\frac{N_c}{N_{eff}}\right)^{\alpha_N} + \left(12B_*S_c \frac{N_{eff}}{L(N_{eff}, C)^{1/\alpha_B} C}\right)^{\alpha_S}$$

$$\frac{1}{C} \approx \frac{\left(L(N_{eff}, C) - \left(\frac{N_c}{N_{eff}}\right)^{\alpha_N}\right)^{1/\alpha_S} L(N_{eff}, C)^{1/\alpha_B}}{12B_*S_c N_{eff}}$$

Substituting $L(N_{eff}, C) \approx \left(1 + \frac{\alpha_N}{\alpha_S}\right) \left(\frac{N_c}{N_{eff}}\right)^{\alpha_N}$, the following equation is obtained.

$$C \approx \frac{12B_* S_c N_{eff}}{\left[\left(\frac{N_c}{N_{eff}}\right)^{\alpha_N} + \frac{\alpha_N}{\alpha_S} \left(\frac{N_c}{N_{eff}}\right)^{\alpha_N} - \left(\frac{N_c}{N_{eff}}\right)^{\alpha_N}\right]^{1/\alpha_S} \left[\left(1 + \frac{\alpha_N}{\alpha_S}\right) \left(\frac{N_c}{N_{eff}}\right)^{\alpha_N}\right]^{1/\alpha_B}}$$

$$C \approx \frac{12B_* S_c N_{eff}}{\frac{\alpha_N}{\alpha_S} \left(\frac{N_c}{N_{eff}}\right)^{\frac{\alpha_N}{\alpha_S}} \left(1 + \frac{\alpha_N}{\alpha_S}\right)^{\frac{1}{\alpha_B}} \left(\frac{N_c}{N_{eff}}\right)^{\frac{\alpha_N}{\alpha_B}}}$$

$$C \approx \frac{12B_* S_c N_{eff}}{\frac{\alpha_N}{\alpha_S} \left(1 + \frac{\alpha_N}{\alpha_S}\right)^{\frac{1}{\alpha_B}} \left(\frac{N_c}{N_{eff}}\right)^{\alpha_N \left(\frac{1}{\alpha_S} + \frac{1}{\alpha_B}\right)}}$$

$$N_{eff}^{\alpha_N \left(\frac{1}{\alpha_S} + \frac{1}{\alpha_B} + \frac{1}{\alpha_N}\right)} \approx \frac{\frac{\alpha_N}{\alpha_S} \left(1 + \frac{\alpha_N}{\alpha_S}\right)^{\frac{1}{\alpha_B}} N_c^{\alpha_N \left(\frac{1}{\alpha_S} + \frac{1}{\alpha_B}\right)}}{12B_* S_c} C$$

Letting $\alpha_C = 1/(1/\alpha_S + 1/\alpha_B + 1/\alpha_N)$, the following equation is obtained.

$$N_{eff} \approx \left[\frac{\frac{\alpha_N}{\alpha_S} \left(1 + \frac{\alpha_N}{\alpha_S}\right)^{\frac{1}{\alpha_B}} N_c^{\alpha_N \left(\frac{1}{\alpha_S} + \frac{1}{\alpha_B}\right)}}{12B_* S_c} C \right]^{\frac{\alpha_C}{\alpha_N}}$$

The following steps simplify the expression for estimating N_{eff}

$$N_{eff} \approx N_c^{\frac{\alpha_C}{\alpha_N} + \alpha_C \left(\frac{1}{\alpha_S} + \frac{1}{\alpha_B}\right)} \left[\frac{\frac{\alpha_N}{\alpha_S} \left(1 + \frac{\alpha_N}{\alpha_S}\right)^{\frac{1}{\alpha_B}}}{12B_* S_c N_c} C \right]^{\frac{\alpha_C}{\alpha_N}}$$

$$N_{eff} \approx N_c^{\alpha_C(\frac{1}{\alpha_S} + \frac{1}{\alpha_B} + \frac{1}{\alpha_N})} \left[\frac{\frac{\alpha_N}{\alpha_S} \left(1 + \frac{\alpha_N}{\alpha_S}\right)^{\frac{1}{\alpha_B}}}{12B_*S_cN_c} C \right]^{\frac{\alpha_C}{\alpha_N}}$$

$$N_{eff} \approx N_c \left[\frac{\frac{\alpha_N}{\alpha_S} \left(1 + \frac{\alpha_N}{\alpha_S}\right)^{\frac{1}{\alpha_B}}}{12B_*S_cN_c} C \right]^{\frac{\alpha_C}{\alpha_N}}$$

$$\frac{N_{eff}}{N_c} \approx \left[\frac{\left(1 + \frac{\alpha_N}{\alpha_S}\right)^{\frac{1}{\alpha_B}}}{12B_*S_cN_c \frac{\alpha_S}{\alpha_N}} C \right]^{\frac{\alpha_C}{\alpha_N}}$$

$$\frac{N_{eff}}{N_c} \approx \left[\frac{\left(1 + \frac{\alpha_N}{\alpha_S}\right)^{\frac{1}{\alpha_S} + \frac{1}{\alpha_B} + \frac{1}{\alpha_N}}}{12B_*S_cN_c \frac{\alpha_S}{\alpha_N} \left(1 + \frac{\alpha_N}{\alpha_S}\right)^{\frac{1}{\alpha_S} + \frac{1}{\alpha_N}}} C \right]^{\frac{\alpha_C}{\alpha_N}}$$

$$\frac{N_{eff}}{N_c} \approx \left[\frac{C}{12B_*S_cN_c \frac{\alpha_S}{\alpha_N} \left(1 + \frac{\alpha_N}{\alpha_S}\right)^{\frac{1}{\alpha_S} + \frac{1}{\alpha_N}}} \right]^{\frac{\alpha_C}{\alpha_N}} \left(1 + \frac{\alpha_N}{\alpha_S}\right)^{\frac{1}{\alpha_N}}$$

Letting $C_c = 12B_*S_cN_c \frac{\alpha_S}{\alpha_N} \left(1 + \frac{\alpha_N}{\alpha_S}\right)^{\frac{1}{\alpha_S} + \frac{1}{\alpha_N}}$, the following equation is obtained.

$$\frac{N_{eff}}{N_c} \approx \left(\frac{C}{C_c}\right)^{\frac{\alpha_C}{\alpha_N}} \left(1 + \frac{\alpha_N}{\alpha_S}\right)^{\frac{1}{\alpha_N}}$$

The loss achieved after compute-efficient training was previously estimated with $L(N_{eff}, C)$,

where the dependence on N_{eff} can now be eliminated as follows.

$$\frac{N_{eff}}{N_c} \approx \left(\frac{C}{C_c}\right)^{\frac{\alpha_C}{\alpha_N}} \left(1 + \frac{\alpha_N}{\alpha_S}\right)^{\frac{1}{\alpha_N}}$$

$$\left(\frac{N_{eff}}{N_c}\right)^{\alpha_N} \approx \left(\frac{C}{C_c}\right)^{\alpha_C} \left(1 + \frac{\alpha_N}{\alpha_S}\right)$$

$$\left(\frac{N_c}{N_{eff}}\right)^{\alpha_N} \approx \left(\frac{C_c}{C}\right)^{\alpha_C} \left(\frac{1}{1 + \frac{\alpha_N}{\alpha_S}}\right)$$

Substituting the above expression into the previous equation for $L(N_{eff}, C)$ provides the following equation.

$$L(N_{eff}, C) \approx \left(1 + \frac{\alpha_N}{\alpha_S}\right) \left(\frac{N_c}{N_{eff}}\right)^{\alpha_N}$$

$$L(C) \approx \left(1 + \frac{\alpha_N}{\alpha_S}\right) \left(\frac{C_c}{C}\right)^{\alpha_C} \left(\frac{1}{1 + \frac{\alpha_N}{\alpha_S}}\right)$$

$$L(C) \approx \left(\frac{C_c}{C}\right)^{\alpha_C}$$

The approximation sign is used in this and the preceding equations due to the estimation of the compute with the compute-bound matrix-matrix multiplication operations. In summary, the above equations provide estimates for the test loss and the number of parameters of the compute-efficient training run given a fixed amount of compute C . Given an estimate of $L(C)$, the critical batch size B_{crit} can also be estimated at $L(C)$.

Additional comments on the theoretical interpretation of the scaling laws

Estimating the critical batch size based on the test loss was an enabling step in the present study. As shown in the previous study by McCandlish et al., the noise scale required knowing the gradients computed across the entire training set at an optimization step, and was not practical for the purposes of the scaling laws. The estimate of the simple noise scale, an approximation of the noise scale, could be computed from a batch, but was less accurate than the noise scale.

The scaling laws are referred to as empirical. However, the definition of an optimal training run is based on the critical batch size. A theoretical derivation for the noise scale based on gradients at an optimization step (with assumptions) was provided in the previous study by McCandlish et al.. They also experimentally demonstrated an approximate correspondence between the estimated simple noise scale and the critical batch size across performance levels, models, and tasks. Therefore, the noise scale provided, at least to some extent, a theoretical interpretation of the definition of an optimal training run in the present study. This definition was also used to define compute-efficient training.

The fact that significant effort was previously invested in the theoretical derivation and validation of the noise scale, suggests that the relation between the test loss and the critical batch size was not known or sufficiently understood at that time. It is this relation, however, that enables to estimate the critical batch size for the test loss values, for which experiments to obtain gradients would not be practical.